

Article

Automated Generation and Evaluation of Interactive-Fiction Serious Games with Open-Weight LLMs

Finn Rogosch *  and Andreas Schrader 

Institute of Telematics, University of Lübeck, 23562 Lübeck, Germany; andreas.schrader@uni-luebeck.de

* Correspondence: finn.rogosch@uni-luebeck.de

Abstract

This work investigates whether open-weight large language models can automatically generate runnable and educationally faithful serious games in a constrained, text-only interactive-fiction (IF) setting. The target games are station-based single-player serious games for knowledge assessment, implemented as IF in a structured, machine-readable text format, and used here as a first step towards later ambient scenarios. A fully automated pipeline called SINE (Serious Interactive Narrative Engine) is evaluated with four prompting strategies, grammar-guided decoding, deterministic validation, and a repair agent. Across a staged evaluation with 240 seeds and increasing complexity, finalist configurations reach success rates between roughly 68% and 86% on the joint criterion of compilation, playability, and learning-goal fidelity. Repair iterations proved central to robustness, whereas grammar masking on top of reasoning prompts did not consistently improve outcomes. The study provides a reproducible benchmark setup, open artifacts, and a constrained generation pipeline as a basis for later extensions toward broader serious game scenarios.

Keywords: serious games; interactive fiction; large language models; constrained decoding; grammar masking; automated validation

1. Introduction

Serious games are understood as digital games that are developed for purposes beyond pure entertainment and are intended to support learning or training [1]. Review studies emphasize the potential of serious games to engage learners and support learning or training goals [1,2]. Challenges in serious-game development include building the game itself (mechanics, content, design, etc.) and connecting game elements to learning goals [3]. Unlike large commercial games, serious games are often developed by smaller teams or by end users themselves [3]. Many teachers have extensive pedagogical expertise but limited programming and game design skills required to design and implement such games. This gap between pedagogical intent and technical skills can substantially hinder the adoption and integration of game-based learning in educational settings because the development process is too demanding and resource-intensive for many teachers [2]. From an educational-technology perspective, this bottleneck is also an access problem because game-based formats may remain unavailable when authoring capacity is limited [2,3]. Automated generation can lower this barrier by turning existing question pools into editable game drafts. Whether such drafts actually support learning still has to be assessed with users.



Academic Editors: Yasmin Hernandez, Karina Figueroa and Maria Elena Chavez-Echeagaray

Received: 20 February 2026

Revised: 16 March 2026

Accepted: 17 March 2026

Published: 18 March 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

Ambient serious games extend this concept by linking learning content to location- and context-dependent interactions in smart environments [4]. Game mechanics are not restricted to a single end device but are embedded in the physical learning environment and use contextual information such as location, time, available artifacts, or environmental state. The aim is to integrate digital game mechanics into learners' everyday activities in an unobtrusive way, with technology ideally perceived only peripherally while learning is supported in context. Based on these requirements, ambient integration also substantially increases development and integration complexity because, in addition to the aspects above, sensing, device orchestration, infrastructure integration, and adaptive mechanisms must be considered.

The present work examines whether large language models (LLMs) can support the automated generation of a deliberately constrained, purely text-based game type. As a starting point, a choice-based interactive-fiction (IF) format without graphical assets is considered, which aligns with text-based generation tasks for LLMs [5]. In SINE, the term narrative is used as an umbrella label for future extensions in interaction modality and deployment context, while the present study evaluates only a controlled IF-based, text-only serious-game scope. Accordingly, the next step is not a change of game purpose, but a shift in modality and context: from text-only serious games to station-based ambient serious games with spatial and context-aware interaction in physical environments. Here, station-based refers to a game structure in which learners move between stations and solve tasks there. The games generated in this work are textual abstractions and serve as a basis for later links to real locations and environmental states. Section 2 positions the work in prior research on LLM-based game and narrative generation. Section 3 defines the objective, research questions, and contributions. Background and methodology are presented in Sections 4 and 5. The empirical findings are reported in Section 6 and interpreted in Section 7. Section 8 closes the paper with a summary and outlook.

2. Related Work

The following section reviews selected work on LLM-based generation of games and narratives and positions the present study within that literature. Generative artificial intelligence (GAI) can support prototyping and development of serious games. LLMs can, for example, take on roles such as designer, narrator, non-player character (NPC), or related functions [5]. The field of Automated Game Design (AGD) studies how new games can be generated by computational methods, often using evolutionary, search-based, or logic-planning approaches [6,7]. The AGD literature includes different machine learning (ML) and search approaches, such as search-based rule generation, random-forest approaches, or conceptual expansion of existing game designs [8–10]. Deep learning methods, including LLM-based approaches, have made substantial progress in adjacent procedural content generation (PCG) tasks such as level generation [11,12]. Prior work also shows that, through in-context learning, LLMs can solve many tasks from instructions and a few prompt examples alone, without additional data-intensive training [13].

Pretrained LLMs provide an alternative entry point. They contain broad world knowledge and can be adapted to specific tasks through fine-tuning or prompt engineering. Fine-tuning is resource-intensive and requires access to model weights, which are not available for many proprietary systems. Prompt engineering is therefore a largely model-agnostic approach in which so-called in-context learning, the direct inclusion of task descriptions and few-shot demonstrations in the prompt, specifies how inputs should be interpreted and outputs structured [14].

Yang et al. [15], in their scoping review of GPT-based methods for games ($n = 177$), show that fully runnable games are less common than partial tasks such as idea generation

or prototyping. The authors explicitly state that, so far, there have been no attempts to concretely combine rule and narrative generation.

In the context of interactive narratives, Zhou et al. show with STORY2GAME how runnable IF games with executable code can be generated from a story [16]. Nasir et al. build a 2D game world around an LLM-generated story [17].

In AGD, several LLM-based approaches presented in recent years focus primarily on rule systems. For example, LLMGG shows that game rules and levels can be generated jointly as a game description in the Video Game Description Language (VGDL) from textual prompts [18,19]. Tanaka & Simo-Serra pursue a similar approach for generating textual game descriptions [20]. Grammar-based decoding is used to generate game descriptions in the Ludii language [21], which yields high compilation rates but does not automatically guarantee semantic correctness. A case study uses ChatGPT (GPT-3.5) as a co-creator for game-design decisions [22]. In that study, three prototypes were developed in each of three genres (baseline, human-extended, and ChatGPT-based) and then evaluated blindly in a user study. The human-extended games are ranked highest in nearly all criteria, while ChatGPT variants are partly perceived as creative but overall do not match the human designs [22]. Other work uses ChatGPT as an evolutionary engine for collaborative game design in which the LLM proposes new design variants and humans evaluate their fitness. In the evaluation, the authors report positive feedback on novelty but identify similarity and incoherence as central weaknesses [23].

In parallel, a growing body of literature examines LLMs as narrators and story generators. Ammanabrolu et al. show how playable IF games can be generated from movie plots by transforming narratives into interactive scenes with an LLM [24]. Sun et al. present 1001 Nights, a system that generates IF scenes live with GPT-4 and additionally uses text-to-image methods for visual enrichment [25]. Farrell & Ware combine a symbolic planner (Sabre) with an LLM that evaluates possible plot continuations for plausibility and thereby guides search in the narrative space [26]. Further work targets specific subproblems: van Stegeren & Mysliwiec use fine-tuned GPT-2 for NPC dialogue generation in quest systems [27], Ashby et al. use GPT-3 in a grammar-based Procedural Quest Generator [28], and Al-Nassar et al. combine BERT and GPT-2 in QuestVille to generate adaptive quests [29]. A common property across many of these studies is that they focus primarily on narrative quality and coherence, while structured validation is often limited to simple plausibility checks.

Some recent approaches try to address rules and story jointly. Zhou et al. propose with STORY2GAME a pipeline that extracts locations, objects, and tasks from a story and generates runnable IF games with executable action code in a Python-based game engine. In evaluation, they report a semantic success rate of about 60% for dynamically generated action code (based on 90 actions) [16]. PANGeA by Buongiorno et al. uses a blueprint for role-playing game phases and lets an LLM enrich both story elements and enemy stats and win conditions, while validation is performed through a validation system that generates gameplay rules from designer criteria and checks inputs and outputs in a self-reflective loop [30]. Lottery & Sprint by Torii et al. [31] describes iterative design sprints with AutoGPT that include reflection steps on rule clarity, win conditions, and balancing or fairness, and their evaluation shows that balancing and fairness remain challenging despite game-master adjustments. Overall, these studies show that LLMs can be used for both narrative and rule drafts, while it remains open how syntactic correctness, playability, and fidelity to intended content can be ensured reliably and automatically.

Multiple review papers on LLM-based co-creation in game design emphasize that specialized roles or agents are useful for different tasks such as story drafting, balancing, and debugging [5,15]. Multi-agent approaches are discussed as a relevant trend, especially

in [5]. At the same time, coherence between outputs of different agents is described as a key challenge, particularly when story, rules, and learning goals are tightly coupled. Prior work such as STORY2GAME [16] and other pipeline approaches therefore uses structured input formats (story sketches, quest skeletons, or JSON-based seeds) as links between generation steps.

STORY2GAME, PANGeA, and Lottery & Sprint each address partial problems, such as automatic game or code generation, validation of free inputs, and human-in-the-loop iteration, but they are primarily oriented towards general IF, role-playing game, or boardgame scenarios rather than serious games. Prior work includes evaluation examples for compilability, playability, and rule validation [16,30,31]. For serious games, however, a fully automated and reproducible evaluation is still missing. The present work addresses this gap.

3. Concept, Objective, and Research Questions

The objective of this work is the automated generation of IF serious games (IF-SGs) based on LLMs. In the long term, this work is embedded in a broader top-down research program that targets automatic generation of station-based ambient serious games, in which learners move through real spaces and solve tasks at distributed stations.

Because ambient serious games span an extremely large search space due to multimodality (space, sensing, interaction, narrative, rule system), a stepwise approach is appropriate, starting with a clearly constrained game type. Building on the infrastructure and game format described by Brandl and Schrader [4], the ambient serious games considered here are single-player knowledge-assessment games in which learners solve multiple-choice tasks at stations in physical space. As a first step, this work therefore focuses on the automated generation of comparable games in an abstracted, purely text-based form. These are station-based single-player serious games for knowledge assessment implemented as IF in a structured, machine-readable text format. Ambient aspects (physical location and sensor data) can be linked to stations as metadata and integrated with the infrastructure in later work.

A central design objective is to reduce the technical burden for teachers so they can focus on didactic content design. Learning content is provided as multiple-choice questions. It is represented in so-called seeds as structured JSON data. The term seed denotes a machine-readable specification of stations and tasks that serves both as input for LLM generation and as a basis for later automated validation.

Given this constrained target setting, the technical scope is intentionally limited. This work deliberately focuses on the narrative layer. Complex rule systems in the sense of general board or card games are not addressed.

Tanaka & Simo-Serra [20] show the value of grammars for domain-specific languages in game generation by reporting a grammar parsability of 68%, meaning that 68% of generated game descriptions are syntactically parseable by a parser based on the Ludii grammar. Empirical work shows that even lightweight validation layers, such as grammar-based decoding or runtime semantic self-reflection validators, can substantially increase output validity or rule compliance of LLM-generated content (for example, 28%→98% in PANGeA and grammar parsability 23%→68% in Tanaka & Simo-Serra) [20,30]. This motivates the investigation of improvements from grammar masking and downstream structured validation in the present context.

Automated generation of serious games with narrative and rule systems creates a very large output space that is difficult to evaluate. One way to constrain the search space is to start from existing games and develop new ones through evolutionary strategies. GAVEL [32], for example, uses a dataset of game descriptions as a starting point and tries

to generate new games in an evolutionary process with LLM mutations. The approach in this work addresses the challenge differently. Instead of primarily relying on evolutionary exploration, the generation process is controlled by combining the constrained game type (station-based single-player quiz games), detailed seed descriptions, formal grammars for the selected target text format, and the validation mechanisms described above (grammar masking, structured checks, and an LLM fixer agent). In this context, SINE uses JSON-based seeds (see Listing 1) and grammar-guided decoding to constrain the LLM search space and structurally safeguard coherence between narrative and learning tasks.

This work addresses three research questions:

1. To what extent can open-weight LLMs generate runnable, structurally valid, and playable IF-SG from a given seed that integrates the specified learning tasks completely and correctly? (RQ1)
2. How do generation strategies, including prompting, reasoning-process configuration, and grammar-constrained decoding, affect generated-game quality in terms of compilation rate, structural validity, playability, learning-goal fidelity, and repair effort? (RQ2)
3. How do seed complexity and model choice (model family and size) affect these quality metrics and the required repair effort? (RQ3)

This work makes three contributions:

1. SINE is presented as a fully automated pipeline that generates station-based IF serious games (IF-SG) from structured seeds into an executable interactive-text format. The pipeline combines grammar masking, structured validation, and an LLM-based fixer agent to produce offline-executable games.
2. Four prompt strategies (S1–S4, see Section 5.2) and open-weight models are systematically evaluated using automated criteria. In addition to compilation rate, structural validity, and playability, the evaluation checks whether a game is completable to the end and whether the specified learning tasks are transferred completely and without content changes (learning-goal fidelity). From a narrative perspective, only playability is considered. A semantic quality analysis of generated text is outside the scope of this work.
3. Seeds, generated games, metrics, and analysis artifacts are published as a reference framework for future work.

Related studies address, for example, the generation of formal game descriptions or rule representations (game description generation) [20,32], or open interactive narratives with emphasis on consistency or playability [16,25,30]. In contrast, this work addresses robust generation and fully automated verification of runnable IF-SG across the full pipeline with verifiable learning-goal fidelity.

4. Background

Serious games can be understood as games with an explicit educational, training, or behavioral purpose in which entertainment is used instrumentally. Taxonomies structure them along purpose and design dimensions [33,34]. Especially in narrative-driven serious games, a central question is how to design action spaces that enable agency while supporting intended learning or training goals. Interactive narrative places user action at the center. Actions influence the course of a dramatic story, while narrative coherence remains a design requirement [35]. Digital storytelling describes multimodal storytelling based on topic selection, research, and scripting, and automatic story generation combines methods that algorithmically generate event sequences for educational and entertainment purposes, among others [36,37]. For ambient-embedded scenarios, work in digital storytelling shows

that text-based narratives can be sufficient to create spatial immersion, for example, when stories are tied to GPS locations (StoryPlaces) [38] or when mobile heritage apps such as Narralive [39] guide users through urban districts [40]. In addition, narrative content is spatially anchored in augmented reality (AR) or virtual reality (VR) contexts, for example, when news stories are told in a VR simulation of an AR news service [41].

For a formal description of narrative and interactive content, a wide range of domain-specific languages (DSLs) has been proposed [42]. Research-oriented DSLs such as SAGA, ATTAC-L, Ceptre, or ScriptEase often target specific narrative or quest structures, while practice-oriented authoring and scripting approaches such as Ink (choice-based IF) or Tracery (author-focused generative text) make interactive stories accessible to a broader audience [40,42]. Many of these languages provide limited capabilities for representing rule aspects (for example, simple state variables or conditions) [42]. Conversely, rule-oriented languages such as GDL, RBG, VGDL, or Ludii allow embedding narrative elements, for example, as metadata or comments [18,21,43,44]. For the present task, however, a combination of sufficiently flexible narrative representation and strictly controllable structure is more important than full coverage of general game mechanics. The concrete DSL used in this work is Ink, a language for creating choice-based games [45,46]. Ink allows modeling branching narratives through text and choices in a deliberately lightweight markup notation.

5. Methodology

The generation and analysis process follows the pipeline shown in Figure 1. Seeds are first defined as the content and structural context for the serious games to be generated. Interactive-fiction scripts with embedded learning tasks are then generated and automatically validated. Faulty outputs are repaired when needed. Finally, the resulting artifacts are evaluated quantitatively using defined metrics. The technical implementation uses the open-source SINE framework [47]. For local inference and grammar masking, SINE uses llama.cpp, integrated via the Python binding llama-cpp-python 0.3.16 [48]. Ink is used as an output format, where narrative and learning tasks are expressed as script text. Ink scripts are compiled with inklecate (ink v1.2.0) using a local binary embedded in the implementation artifact. A complete Ink script generated during the experiment, serving as an example of the syntax used, is shown in the Appendix A in Listing A1.

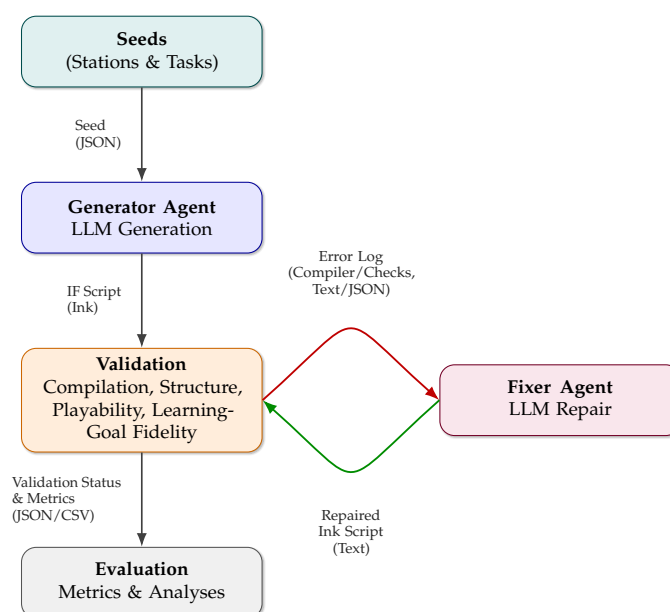


Figure 1. Overview of the generation and evaluation process. Seeds are used as input for the generator agent, followed by validation and an optional fixer agent, resulting in final evaluated serious games.

5.1. Seed

As the basis for game generation, a seed describes immutable context information for the game to be generated. In the version used here, this information includes:

1. possible stations (locations) with their IDs;
2. learning content to be conveyed as multiple-choice questions (tasks).

Seeds are randomly assembled from two topic areas (media and medicine) and used in four rounds with increasing complexity. In the experiments reported here, seeds contain between 2 and 15 stations and between 2 and 15 tasks. The lower bound avoids trivial instances, and the upper bound keeps runtime and error rates within a practical range. Both bounds are identical to keep complexity effects comparable across station structure and task volume. A total of 240 unique seeds is used. Rounds A–C serve as compact screening (20 seeds each), and Round D serves as the main evaluation (180 seeds). A seed example from the experiment is shown in Listing 1. Seed instances are sampled from topic-specific station and multiple-choice task pools with stable IDs and fixed task order to support consistent downstream checks. A fixed round protocol preserves benchmark reproducibility after sampling. A full factorial design was not used because the resulting number of seed combinations would have increased substantially and would have made the benchmark much more expensive to compute. It would also have produced extreme station-task combinations that would overemphasize either narrative scaffolding or quiz content and were therefore considered didactically implausible.

Listing 1. Seed example from the experiment (seed seed_media_13_t2_006).

```
{
  "locations": [
    {"id": "location_s003"},
    {"id": "location_s013"},
    {"id": "location_s005"}
  ],
  "tasks": [
    {
      "id": "task_media_008",
      "seq": 1,
      "question": {
        "stem": "Which color model is primarily used for printing?",
        "options": ["RGB", "CMYK", "HSV", "Lab"],
        "correct": 1
      }
    },
    {
      "id": "task_media_045",
      "seq": 2,
      "question": {
        "stem": "What does VR stand for?",
        "options": ["Video Recording", "Virtual Reality",
          "Variable Rate", "Visual Rendering"],
        "correct": 1
      }
    }
  ]
}
```

5.2. Generation

Games are generated exclusively with open-weight models that run locally via llama.cpp, ranging from compact 8B models to 32B models. To limit variance and ensure comparable conditions, the same sampling profile is used for generator and fixer agents. Decoding uses *temperature* = 0.7, *top_p* = 0.9, and deactivated *top_k* = 0. This parameterization follows model-related recommendations in the Unsloth documentation [49] and was considered suitable in informal pilot tests. A lower temperature could, in principle, further increase compilation probability for the domain-specific language.

Model selection is driven by the technical requirement to use runtime grammar masking. At development time, this functionality was available only through llama.cpp with the GGUF quantization format [50], which constrained selection to variants that had been converted accordingly. To ensure broad coverage of current open-source models, GGUF versions were sourced via Hugging Face [51]. The primary source was the community provider Unsloth [52], which provides a consistent model portfolio at comparable quantization levels.

The focus was on models with parameter sizes between 8B and 32B, representing a compromise between inference speed on commodity hardware and sufficient model capacity for complex generation tasks. In pilot tests, several model families were excluded for technical reasons. GPT-OSS [53] and IBM Granite 4.0 [54] could not be integrated consistently into the pipeline workflow due to incompatible Jinja chat templates. Microsoft Phi 4 [55] could not be executed successfully in any tested configuration. DeepSeek models [56] were identified as established and capable but clearly exceeded the hardware limit used here due to minimum sizes of 70B+, and were therefore not evaluated further.

The final model selection includes representatives of the Gemma 3 [57], Llama 3 [58], Qwen 3 [59], and Magistral [60] families, which showed stable inference, compatible chat templates, and sufficient generation quality in pilot tests. Q8 quantization was consistently used across all models. Compared with FP16, it reduces memory demand and typically leads to lower-quality degradation than aggressive 4- or 5-bit quantization [61]. Context-window size and generation limits are described below.

Four strategies vary in prompt structure, in particular whether an explicit planning section is included, and the degree of syntactic guidance:

- S1 (no reasoning): direct output without an explicit planning section.
- S2 (reasoning optional): short, optional planning section before the story.
- S3 (reasoning required): mandatory planning section for structural planning.
- S4 (grammar): as S3, but additionally guided by an Ink GBNF formal grammar.

S1 serves as a one-shot baseline with direct Ink output. S3 tests whether an explicitly required planning phase improves structural consistency. S2 was included as a middle ground because some models appear to invoke such planning only when it seems useful, so optional reasoning tests whether this flexibility is preferable to forcing it in every run. S4 adds grammar-constrained decoding on top of S3 because prior work suggests that grammar guidance and validation layers can improve output validity and rule compliance [20,30]. Because an explicitly required planning phase was hypothesized to be the strongest prompt-only setup for structurally demanding outputs, the additional grammar layer was tested on top of S3 as the most plausible place for further gains while keeping the comparison practically manageable.

The generator agent is an LLM that generates an initial story script in Ink from the seed, including embedded learning tasks. System instructions (shared core plus strategy-specific additions) define output format, consistency requirements, and Ink syntax. The system message is assembled at runtime from multiple component groups. Composition follows a fixed order of sections, each with a clearly separated function.

1. Shared rules: Global quality and format requirements are set first and apply to all runs.
2. Agent task: The concrete assignment for the current phase is then set depending on whether the agent is a generator or a fixer.
3. Strategy profile (S1–S4): The selected strategy then controls whether and how strongly explicit planning steps are required before the story.
4. Optional guidance: Examples and adaptive feedback can additionally be shown to reduce recurring errors.
5. Final checks: Mandatory final checks are appended before output is passed to the model.

The user message is built separately and contains the seed as JSON for the generator and, for the fixer, additionally an error log and the latest story version. In the grammar strategy, the call is also constrained by a formal grammar. For space reasons, the full prompt texts are not printed in the appendix and are documented in the SINE implementation [47].

Optional or mandatory reasoning phases structure the content and formal planning of the story. Conceptually, this can be understood as prompting where inputs are transformed into text prompts via templates and instructions, and a language model completes them probabilistically [14]. For narrative tasks, multi-stage prompt setups are often used to separate content generation from formal structuring. GENEVA, for example, first generates branching storylines under predefined constraints and then outputs them as graph code in a second step [62]. Interactive narrative systems also use separate prompt phases for state checks, for example, a binary decision about whether a trigger condition is satisfied in the current script [63]. Because prompt-engineering methods increase the probability of syntactically correct outputs but cannot guarantee syntactic correctness, S4 extends instructions with grammar-based guidance as provided by constrained decoding with context-free grammars [64]. The Ink GBNF formal grammar used in S4 is shown in the Appendix B in Listing A2. It enforces a <think> block, followed by an Ink script wrapped as ````ink`, and an explicit end-of-file marker. The grammar intentionally covers only the subset of Ink needed in SINE. Language constructs not required in this project, for example, specific functions, are not included in the grammar. Accordingly, Ink syntax in prompting is also described only for this subset. Both the Ink example and the Ink syntax explanation in prompting are restricted to this subset.

The fixer agent is activated only when compilation or validations fail. It receives the seed, the most recent generated story version, and a log of observed errors, for example, missing nodes, inconsistent branches, or incorrect answer options. Repair, therefore, relies on reliable external feedback from deterministic checks instead of self-feedback produced by prompting alone. This aligns with the observation that self-correction is reliable mainly when external feedback signals are available, whereas prompted LLM feedback in general tasks often does not yield consistent improvements [65]. The number of repair attempts is limited to three fixer iterations. Together with the initial generator call, this yields at most four total LLM iterations per run ($I \in \{1, 2, 3, 4\}$), which controls resource consumption and avoids artificially inflated success rates through many repetitions [65]. Pilot tests showed asymptotically decreasing gains in success rate per additional fixer iteration, so three fixer iterations were set as a compromise between success rate and runtime.

Each combination of seed, model, and strategy follows the same process: (1) generation of an initial Ink script by the generator agent, (2) compilation test, (3) automated validations and metric computation (see Section 5.3), and (4) up to three repair attempts by the fixer agent if a step fails. All artifacts and metadata are logged per run, enabling analyses at the individual-run, seed, model, and strategy levels.

5.3. Evaluation

Evaluation is fully automated. The metrics used and their operationalization are defined below. This follows the approach of objective metrics for interactive narratives that can be derived from interaction or system traces, provide quantitative feedback with low manual analysis effort, and can complement potential questionnaires or interviews [66]. A run is a combination of seed, model, and strategy. Results are aggregated across runs and reported separately by model, strategy, and seed complexity. Let N be the set of all runs considered in such a cell (for example, all seeds of one model and one strategy).

Primary Metric: Success.

For each run $i \in N$, three boolean predicates are defined and encoded as 0/1 variables. Compilation (C_i), playability (P_i), and learning-goal fidelity (Q_i) are considered, where $C_i, P_i, Q_i \in \{0, 1\}$. Success is the logical AND of these three predicates. Under 0/1 encoding, this AND can be written as a product:

$$S_i = C_i \cdot P_i \cdot Q_i. \quad (1)$$

Thus, $S_i = 1$ holds exactly when all three conditions are met. A run is successful if $S_i = 1$. The success rate of a cell is

$$r_{\text{succ}} = \frac{1}{|N|} \sum_{i \in N} S_i. \quad (2)$$

Success was operationalized through three necessary conditions. Compilation means that $C_i = 1$ holds exactly when the generated Ink script is processed without error by the Ink compiler. Compilation is therefore defined as a purely syntactic minimum requirement because only error-free compiled scripts are reliably executable. In practice, the script is converted into an intermediate representation (for example, JSON), which can then be used for additional checks [45]. The compilation rate is

$$r_{\text{comp}} = \frac{1}{|N|} \sum_{i \in N} C_i. \quad (3)$$

Playability means that the generated game forms a playable directed graph of story beats and choices. The playability check, therefore, tests whether at least one path exists from a start node to a terminal ending (END). This is implemented with a simple breadth-first search (BFS) on the choice graph. For non-compiled runs, $P_i = 0$. The playability rate is

$$r_{\text{play}} = \frac{1}{|N|} \sum_{i \in N} P_i. \quad (4)$$

Learning-goal fidelity means that all multiple-choice tasks specified in the seed are represented unchanged in the final story. The criterion is intentionally strict so that content and reference answers preserve the instructor's intention and hallucinated deviations do not communicate incorrect results. Operationalization uses simple syntactic equalities, including an identical number of answer options, trimmed lowercased equality of question and answer texts, and a match of the option marked as correct. This baseline prioritizes robust automated screening and reduces the risk of silent answer-key corruption. This strict operationalization prioritizes content-preserving checks and may therefore penalize semantically equivalent paraphrases. For non-compiled runs, $Q_i = 0$. The corresponding rate is

$$r_{\text{quiz}} = \frac{1}{|N|} \sum_{i \in N} Q_i. \quad (5)$$

Secondary Metrics.

Two additional metric groups are reported. Output form and size are used for comparative characterization of generated outputs, based on simple structural and size features computed from the final script for each run $i \in N$. For non-compiled runs ($C_i = 0$), these features are not computed. Let K_i be the number of nodes and E_i the number of decision points. Normalized density is defined as $\delta_i = E_i / \max(1, K_i)$. Story text size is operationalized as the word count W_i . Structural and choice-space features such as node count, decision points, and decision density are closely related to metric families on length or extent and choice characteristics proposed as objective indicators for interactive narratives [66]. Those are often computed from gameplay log indicators such as duration and choice frequency, whereas this work computes only artifact-based structure and size features of generated scripts.

Resource usage and repair effort are used for efficiency comparison, covering (i) repair effort as the number of model calls or iterations to the final version and (ii) resource metrics for language-model inference. Such efficiency and cost aspects are considered an important complement to pure quality metrics in LLM evaluation surveys [67]. For each run $i \in N$, let I_i be the total number of LLM iterations to the final version (one generator iteration plus zero to three fixer iterations). As an indicator for the use of a repair phase, define $F_i \in \{0, 1\}$ with $F_i = 1$ exactly when $I_i > 1$. Token effort is operationalized as T_i^{in} (prompt tokens), T_i^{out} (generated tokens), and $T_i = T_i^{\text{in}} + T_i^{\text{out}}$. Runtime is measured as D_i in seconds from the difference between run start and end times. Aggregated over N , means and medians are reported for I_i , T_i , and D_i , supplemented by proportion values such as $r_{\text{fix}} = \frac{1}{|N|} \sum_{i \in N} F_i$.

Multi-Stage Experimental Design.

Evaluation is conducted in four rounds (A–D) with increasing seed complexity. In screening rounds A–C, 20 different seeds are used per round, and in main round D, another 180 seeds are used, resulting in 240 unique seeds with 2–15 stations and 2–15 tasks. This split limits the compute and token cost in early iterations. After each round, model-strategy combinations are filtered by success rate. For a given model, only strategies with a success rate of at least 0.75 and at most $\Delta = 0.25$ points below the best strategy of that model are retained. In this way, the full seed set in round D is applied only to configurations that robustly compiled, produced playable stories, and preserved learning-goal fidelity in earlier rounds. Because seed complexity is stratified across rounds, the main quantitative results for finalist configurations in Section 6 are reported in aggregated form over all rounds A–D.

Statistical Uncertainty.

Reported success rates are proportions over binary per-run metrics, in particular r_{succ} as the fraction of runs with $S_i = C_i \cdot P_i \cdot Q_i$ (compilation, playability, learning-goal fidelity). For each model-strategy combination, the design is to run each of the 240 seeds exactly once (rounds A–D). In analyses, N is explicitly reported per cell because individual runs may be excluded from aggregation due to aborts or missing artifacts. As a rough rule of thumb, at $N \approx 240$ the variation width is about ± 6.3 percentage points based on the binomial standard error of a proportion and a normal approximation for a 95% band in the worst case ($p \approx 0.5$). For comparison, related systems also use fixed evaluation suites and report proportion metrics such as parsability, playability, or success rates (for example, 100 instances in [20] and 8 stories per length group in [16]). Significance tests and formal effect sizes are omitted. Accordingly, clear larger gaps are interpreted as robust, while small differences are treated cautiously. Resource and repair metrics are additionally reported (Section 5.3), which reveal practical differences even when success rates are similar.

5.4. Reproducibility

The full experimental setup and resulting dataset are released as open source. Only open-weight LLMs are used in the experiments, specifically models from the Llama 3 family [58] and the Qwen 3 and Gemma 3 series [59]. Evaluated model sizes range from compact 8B models to 32B parameters. All scripts and programs are designed to run on commodity workstation hardware.

Hardware footprint. Experiments were run on comparable Apple Silicon hosts using the Metal Performance Shaders (MPS) backend. The host pool included comparable Apple Silicon systems such as M3 Max-class machines with 128 GB unified memory and Mac Studio-class machines with M1 Ultra and 96 GB shared memory, which enable local execution of the target LLMs without specialized datacenter GPUs. To keep overall runtime manageable, complete runs were distributed across comparable local hosts with identical settings and merged only for consolidated reporting. The released pipeline runs unchanged on a single comparable host, with longer wall-clock times. Memory utilization was not logged as a comparative metric because it depends strongly on host setup, quantization, and parameter count, and therefore added little interpretive value beyond successful local execution. Across all 1920 finalist runs, the mean total runtime is 595 s, and the median total runtime is 459 s, with the mean generation time 204 s, the mean compilation-check time 0.61 s, and the mean fixer time 390 s. For successful finalist runs, the median total runtime is 332 s. On the reported hardware, this corresponds to a few minutes for generating a small set of playable games.

6. Results

The term finalists refers to the eight model-strategy configurations evaluated in round D after multi-stage screening. For Gemma 3 27B and Qwen 3 14B, three strategies are included in each case (grammar, reasoning optional, reasoning required). For Qwen 3 8B and Qwen 3 32B, reasoning optional is included. Table 1 shows their aggregated rates across all rounds A–D. Across all finalists, r_{succ} ranges from 59.6% (Gemma 3 27B, reasoning optional) to 85.8% (Qwen 3 14B, reasoning required). A complete example of an Ink script generated in the experiment is documented in the Appendix A in Listing A1. Figure A1 complements this listing with a reduced beat graph of the same sample. It illustrates how correct answers drive progression while wrong-answer branches are routed into local retry loops that preserve overall playability.

Across all finalist configurations, differences between media and medicine seeds are small (success rate about 77.9% vs. 75.4%).

Table 1. Aggregated rates across all rounds A–D for finalist configurations.

Model	Strategy	r_{comp}	r_{play}	r_{quiz}	r_{succ}
Qwen 3 14B	reasoning required	93.8	92.9	86.7	85.8
Qwen 3 14B	reasoning optional	92.1	91.3	83.8	83.3
Gemma 3 27B	reasoning required	92.5	90.0	80.8	78.8
Qwen 3 14B	grammar	91.7	90.8	78.8	77.9
Gemma 3 27B	grammar	87.5	86.7	75.4	75.0
Qwen 3 8B	reasoning optional	82.1	77.5	70.0	67.9
Qwen 3 32B	reasoning optional	79.2	78.3	67.5	66.7
Gemma 3 27B	reasoning optional	90.8	86.3	63.8	59.6

The contribution of the fixer agent is particularly clear. Considering only the first generator iteration, the mean success rate is about 32.3%. With up to three additional repair attempts (up to four total iterations), it rises to about 74.4% (Figure 2). At least one fixer

step is used in most successful runs. On average, about $\bar{I} \approx 2.3$ LLM iterations are used per run. A fixer phase is activated at all in about 65% of runs, corresponding to $r_{\text{fix}} \approx 0.65$.

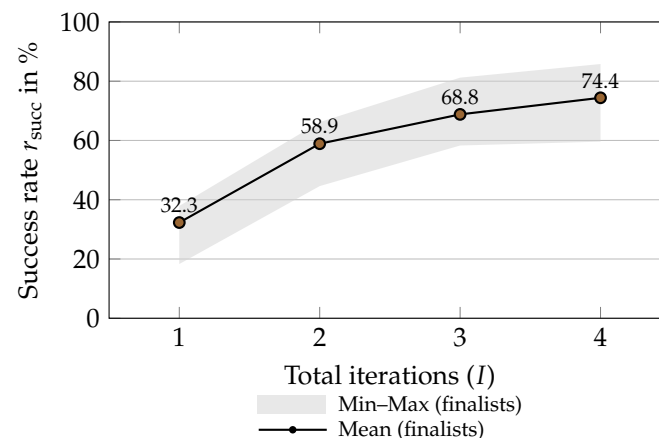


Figure 2. Success rate r_{succ} as a function of total iterations I (one generator iteration plus up to three fixer iterations), shown as mean and Min-Max range across all finalists (aggregated over all rounds A–D).

Failure-tag statistics for finalists show which validation problems occurred at least once during a run (generator and fixer iterations) (Table 2). Despite a failure tag, a run can still be successful if a later fixer step resolves the issue. The table, therefore, describes repair demand and typical stumbling points, not the number of aborted runs. Multiple assignments are possible. Reported shares refer to all 1920 finalist runs.

Table 2. Most frequent failure tags that occurred at least once during a finalist run (multiple assignments possible).

Tag	Category	Count	Share of Runs (%)
compile:compile_failed	compile	768	40.0
quiz:task_mismatch	quiz	690	35.9
quiz_detail:missing_task	quiz_detail	688	35.8
fixer:exhausted	fixer	421	21.9
playable:no_terminal_path	playable	134	7.0
quiz_detail:extra_task	quiz_detail	88	4.6

Across all finalist configurations, success rate decreases with increasing seed complexity (Figure 3). Compilation and playability rates remain substantially higher than the learning-goal fidelity rate (around $r_{\text{comp}} = 71\text{--}100\%$ and $r_{\text{play}} = 68\text{--}98\%$), so the success rate closely follows the learning-goal fidelity rate. The decline is therefore coupled more strongly to learning-goal fidelity than to compilation or playability alone. The same trend appears at the model level. The correlation between seed size and success is negative for all finalist models (for stations about $r = -0.02$ to -0.32 , for tasks about $r = -0.17$ to -0.41). Local non-monotonic dips at individual complexity levels are expected in these binned curves. Per-bin sample size is still substantial (160 or 320 runs), but the seed suite is not fully factorial: numbers of stations and tasks are partly coupled in fixed combinations (for example, four stations only with 7 tasks, and eight only with 12). Rates are therefore marginal bin means over the other complexity axis and should be interpreted as composition effects in addition to the global trend.

Regarding story structure, finalists generate partly extensive and branching stories. Gemma 3 27B (reasoning required) reaches on average about $K \approx 23$ nodes with $E \approx 38$ choices and $W \approx 620$ words of story text. Qwen 3 14B (reasoning required) reaches

about $K \approx 26$, $E \approx 45$, and $W \approx 410$. Qwen 3 8B (reasoning optional) reaches about $K \approx 25$, $E \approx 45$, and $W \approx 340$. Qwen 3 32B (reasoning optional) generates more compact stories with about $K \approx 14$, $E \approx 31$, and $W \approx 190$. Across complexity levels, success rates remain high for very simple seeds (2–3 stations or tasks) and decrease clearly for large seeds (up to 15 stations or tasks). At an aggregate level, model and strategy choices appear to affect narrative pacing. Larger outputs tend to allow broader exploration, whereas more compact outputs reach checkpoints and endings faster.

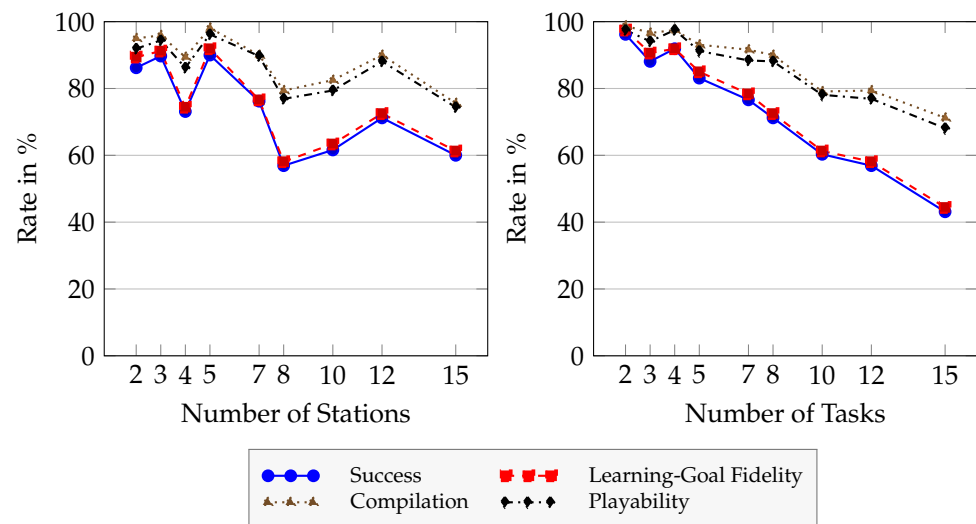


Figure 3. Success, learning-goal fidelity, compilation, and playability rates aggregated over all finalists as a function of seed complexity (stations and tasks). Curves show empirical bin means without smoothing.

7. Discussion

Across the finalists, success rate is limited mainly by learning-goal fidelity, whereas compilation and playability remain high across many seeds. The fixer agent acts as a central stabilization layer because only a few additional iterations compensate a large share of initial failures. The following sections discuss these tradeoffs by comparing grammar-guided decoding and explicit reasoning phases, followed by limitations and scope.

7.1. Interpretation

Results show that the SINE pipeline can automatically generate runnable IF-SG across a broad range of seeds and preserve embedded learning tasks with high accuracy. A success rate between roughly 68% and 86% across compilation, playability, and learning-goal fidelity is substantial, especially for open models run locally. In practical terms, the pipeline already supports prototypical serious-game generation. In the best finalist configuration, $r_{\text{succ}} = 85.8\%$ corresponds to roughly six out of seven runs that produce an executable, playable, and learning-goal-faithful Ink game without human intervention. Regarding RQ1, the results show that open-weight LLMs can generate runnable, structurally valid, and playable IF-SG from structured seeds at a meaningful scale. Reliability, however, still falls short of fully dependable end-to-end automation. The following interpretation then examines where this remaining gap is shaped by strategy effects (RQ2) and by complexity and model effects (RQ3).

Error analyses also reveal the limits of current LLM capabilities. The fixer agent is not an optional add-on but a core pipeline component. Without it, the success rate is 32.3% (finalists, generator iteration only), which means more than half of the seeds fail at least one step in the chain of syntax, structure, and learning-goal fidelity. For unsuccessful runs ($S_i = 0$ after exhausting all fixer iterations), the dominant failure causes are mainly

learning-goal fidelity issues (missing or modified tasks). This highlights the difficulty of preserving all content details of a seed while adhering to strict formatting and grammar constraints. Because compilation and playability rates remain largely high in the examined settings, the success rate is primarily bounded by learning-goal fidelity. Two next steps follow from this result. The pipeline itself can still be improved, for example, through prompt design, model choice, two-stage generation, or specialized fixer agents. In addition, the metric should be extended beyond exact matching so that partial matches and semantic equivalence can be represented more precisely. A direct path towards 100% success is an error-type-specific, tool-augmented repair loop: formal and structural violations are corrected through deterministic micro-operations exposed as tools, while content deviations are handled by LLM-guided template reconstruction with strict target-versus-actual checks against reference data. Specialized LLM fixer agents with domain-specific prompt modules can additionally target individual error classes and raise the overall success rate while avoiding broad rewrites or replacement of entire game sections.

Results further indicate that extending reasoning required with an additional grammar mask does not provide a consistent advantage in this setting. The core idea of formal guidance remains plausible, but for this narrative task with typical model errors, additional masking increases control and token effort without measurable improvements in success or compilation rates, and forced reasoning structure is not equally natural for all model families. Most remaining compilation errors are caused by referential consistency issues such as duplicate node names or missing divert targets, which are only partially covered by a static grammar. Near-complete safeguards would require dynamically constructed run-specific constraints, which adds implementation effort. In direct comparison, both examined models show higher success rates with lower token usage under reasoning required, suggesting that grammar masking in the tested form is better interpreted as a context-dependent option than as a general default strategy. RQ2 is therefore best answered by the interaction of prompting setup, repair iterations, and validation constraints rather than by a single isolated prompting variant. A small spot-check of archived finalist outputs suggests that prompting affects how explicitly progression is represented in the scripts. In inspected S2 runs, optional reasoning more often yielded direct task-centered progression with fewer dedicated transition knots, whereas S3 more often separated room changes, transition scenes, and quiz nodes more explicitly. S4 then tended to preserve this explicit station rhythm while further regularizing local answer-loop patterns and short success beats, but the inspected outputs did not indicate a corresponding improvement in global narrative planning. This reading is also consistent with the story-metric comparison, which indicates differences not only in total size but in how choices are distributed across knots. These observations provide a tentative link between prompt design and output form, but they are not strong enough to support claims about stable model tendencies. Such tendencies can also vary with quantization and sampling settings [68,69].

The negative regression of success rate with increasing numbers of stations and tasks (Figure 3) underlines that robustness is not only a matter of syntactic validity. For larger seeds, the risk increases that individual learning tasks are missing, reordered, or content-distorted, which reduces learning-goal fidelity and, in turn, directly lowers success rate. Complexity analysis shows that the approach remains robust even for seeds with two-digit station counts (up to 15), but success rates decline as expected with growing combinatorics. With respect to RQ3, seed complexity shifts success rates downward, while model family and parameter size influence the steepness of that decline. Interpretation of station-wise regression should still be cautious. The seed suite is not fully factorial, so station and task counts are partly coupled, which can increase the apparent volatility of station bins. This coupling is not only a statistical limitation. Very few tasks across many stations would

inflate narrative scaffolding relative to assessed learning content, whereas many tasks across very few stations would compress the experience into a quiz-heavy interaction pattern. The chosen seed ranges were therefore intended to maintain a plausible balance between narrative framing and learning content, even though this practical restriction makes it harder to obtain isolated causal estimates for station and task counts. In addition, success is currently operationalized via compilation, playability, and task-level fidelity, while explicit station-level coverage is not scored. Because station use in prompting is framed as allowed rather than mandatory, missing or merged stations can remain undetected unless they cause downstream structural failures. Future evaluations should therefore include explicit station-fidelity metrics and use factorial seed designs to more clearly isolate station and task effects. For practical use, two scenarios seem realistic: (1) automatic generation of small to medium games that need little post-editing, and (2) semi-automatic generation of more complex scenarios in which SINE mainly provides structure and draft ideas for later human refinement.

Finally, evaluation is currently limited to structural and content-based metrics. Neither narrative quality nor actual learning effectiveness in context was empirically assessed in this work. A follow-up user study with teachers and students is already planned and will assess content-level comprehensibility, perceived narrative coherence between story and learning tasks, and engagement. A later step will examine actual learning outcomes in deployment settings. A separate later strand will examine how user feedback can be integrated iteratively through co-creation and human-in-the-loop workflows.

7.2. Limitations and Scope

To contextualize the results, this section makes empirical scope explicit and identifies key metric limitations, plus notes on validity and generalizability.

Scope.

Results apply to a deliberately constrained game type: station-based textual single-player IF in Ink format with explicitly specified multiple-choice tasks. SINE is not intended as a general method for arbitrary serious games. It targets a constrained subspace in which structure and learning-goal constraints can be checked automatically. Model selection is also constrained because only open-weight LLMs with local inference are considered. Results may differ for proprietary models, fine-tuning setups, or other inference stacks.

Metric Limitations.

Compilation (Section 5.3) is a necessary syntactic minimum requirement, but it does not ensure game-mechanical or content semantics and does not correct such errors. Playability (Section 5.3) operationalizes playability through the existence of a path to a terminal ending (END) and does not capture gameplay quality, narrative coherence, or content breaks along concrete choice paths. Learning-goal fidelity (Section 5.3) is intentionally defined strictly via text-based equalities. This reduces the risk of silent content distortion, but semantically equivalent reformulations may still be counted as errors, and the didactic quality of tasks is not represented. The evaluation, therefore, measures robustness under formal constraints, not the semantic quality of story texts or learning outcomes in the deployment context. These structural metrics were selected because serious-game design commonly treats the link between learning objectives and gameplay mechanics as a central concern [1,70]. Related interactive-narrative work also suggests that some log-based indicators can correlate with questionnaire-based experience measures [66]. Here, these metrics are used primarily as prerequisite checks. At most, they may serve as proxy indicators of engagement when combined with user data. The strict equality-based screen also

compresses near misses into a single error class, because semantically adequate reformulations and partially correct task transfers are counted alike. Future work should therefore complement the current screen with semantic checks that can distinguish partial matches and meaning-preserving reformulations, for example, embedding-based similarity and task-level verifier judgments.

Notes on Validity and Generalizability.

The seed suite covers a broad complexity range (2–15 stations or tasks), but it remains a specific selection and does not represent the full diversity of real teaching and learning scenarios. External validity remains open, especially for other task types (for example, open responses or simulations), other representations, and multimodal or ambient-embedded games. Internal validity can be affected by non-determinism in LLM inference and prompting. Reported rates are snapshots for the specific configuration of seeds, prompts, models, and hyperparameters. In addition, differences in toolchain and Ink compiler versions can affect observed error classes and success rates. The results support the claim that form and fidelity can be controlled automatically in a constrained setting. They should not be read as evidence for broader narrative or didactic effectiveness.

8. Summary and Outlook

The evaluated pipeline combines structured prompting, automated validation, an LLM-based fixer agent, and a grammar-guided decoding variant. Taken together, the results indicate that structured prompting, automated validation, and the fixer agent enable the generation of executable, playable, and content-faithful serious games in Ink format from compact seeds for a substantial proportion of runs, while grammar-guided decoding serves as a context-dependent additional control layer rather than a consistent overall advantage. Open-weight models such as Gemma 3 and Qwen 3 reach success rates between roughly 68% and 86% across all rounds in the SINE pipeline (finalists), without relying on proprietary cloud infrastructure.

Methodologically, the study shows how multi-agent setups, grammar-guided decoding, and automated validation can interact to make generative systems more robust for critical domains such as education. The proposed metrics and open dataset allow comparative positioning of future pipeline variants, for example, with other models, prompt designs, or repair strategies.

The current setup provides a simplified baseline for subsequent work on ambient serious-game scenarios. From this purely textual, station-based format, the pipeline can be extended stepwise to incorporate spatial grounding, multimodal interaction, and more complex rule systems. Future work should follow three strands. First, stronger coupling of structural planning and learning-goal fidelity should be pursued, for example, through explicit constraint or program-repair techniques. Second, expansion to further domains and task types is planned, for example, open questions, simulations, and ambient-embedded scenarios. Third, user-centered evaluations should be integrated, with teachers and learners assessing games generated by SINE in real deployment contexts. One realistic application would be a seminar-style course in which teachers and students jointly develop game components and combine them into one complete game. The resulting game could be used as structured exam preparation at the end of a semester. Over multiple semesters, such a workflow could produce a growing catalog of domain-specific learning games that can be reused and iteratively improved.

Author Contributions: F.R. led the work, including methodology, implementation, experimentation, and original draft preparation. A.S. contributed to conceptualization, provided supervision, and performed substantial review and editing of the manuscript. All authors have read and agreed to the published version of the manuscript.

Funding: This work was produced in the context of the project *Life Labs*, which is funded by Stiftung Innovation in der Hochschullehre within the funding program *Lehrarchitektur* (grant number 1001-3214).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The software and data supporting the reported results are publicly available via the Zenodo (Version 1.0.0) release for this work [47].

Conflicts of Interest: The authors declare no conflict of interest.

Abbreviations

The following abbreviations are used in this manuscript:

2D	Two-Dimensional
AGD	Automated Game Design
AR	Augmented Reality
ATTAC-L	DSL Name
AutoGPT	Tool or System Name
BERT	Bidirectional Encoder Representations from Transformers
BFS	Breadth-First Search
ChatGPT	GPT-Based Chatbot
CSV	Comma-Separated Value
DSL	Domain-Specific Language
END	Terminal End Marker in Ink
FP16	16-bit Floating Point
GAI	Generative Artificial Intelligence
GAVEL	System Name
GBNF	Grammar Format (BNF Variant)
GDL	Game Description Language
GENEVA	System Name
GGUF	File Format for Quantized Models
GB	Gigabyte
GPS	Global Positioning System
GPT	Generative Pretrained Transformer (e.g., GPT-2/3/4)
GPT-OSS	Model Name
IBM	International Business Machine
IF	Interactive Fiction
IF-SG	Interactive Fiction Serious Game
JSON	JavaScript Object Notation
LLM	Large Language Model
M1	Apple M1 (Chip Generation)
M3	Apple M3 (Chip Generation)
ML	Machine Learning
MPS	Metal Performance Shader
NPC	Non-Player Character
PCG	Procedural Content Generation
Q8	8-Bit Quantization
RBG	Regular Boardgame

SAGA	DSL Name
SINE	Serious Interactive Narrative Engine
STORY2GAME	System Name
VGDL	Video Game Description Language
VR	Virtual Reality

Appendix A. Ink Example

Listing A1. Complete Ink script from the experiment (seed `seed_media_12_t3_004`, Qwen 3 14B, reasoning required).

```

-> location_s007_intro

=== location_s007_intro ===
#scene_hook
The hum of analog tape machines fills the air as you enter the
abandoned sound studio. Faded posters advertise "CD Mastering 1985."
A flickering monitor displays a waveform frozen in time.
* [Examine the mixing console] -> location_s007_console

=== location_s007_console ===
#task_media_067
What is the bit rate of uncompressed CD audio?
* [705 kbps] -> location_s007_failure1
* [1411 kbps] -> location_s007_success1
* [2822 kbps] -> location_s007_failure1
* [5644 kbps] -> location_s007_failure1

=== location_s007_success1 ===
#correct_answer
The waveform resumes, revealing "SAMPLE RATE CONFIRMED."
A drawer in the console clicks open, revealing a keycard.
-> location_s011_intro

=== location_s007_failure1 ===
#incorrect_answer
Alarms blare as the studio's lights strobe red.
The waveform distorts before freezing again.
* [Try another approach] -> location_s007_console

// ... additional task and failure blocks omitted...

=== location_s007_final ===
#scene_hook
Back in the sound studio, you insert the data drive into a terminal.
A holographic interface materializes with a final question:
#task_media_035
What is the typical aspect ratio for cinema screens?
* [16:9] -> location_s007_failure3
* [21:9] -> location_s007_failure3
* [4:3] -> location_s007_failure3
* [2.35:1] -> location_s007_success3

=== location_s007_success3 ===
#correct_answer
The hologram explodes into a cascade of film reels and soundwaves.
A voiceover declares, "ACCESS GRANTED." The walls dissolve into light
as the escape sequence completes.
-> END

=== location_s007_failure3 ===
#incorrect_answer
The terminal emits a low growl, its screen flashing "ERROR: WRONG FORMAT."
The room shakes as seismic sensors activate.
* [Try again] -> location_s007_final
#end_of_file

```

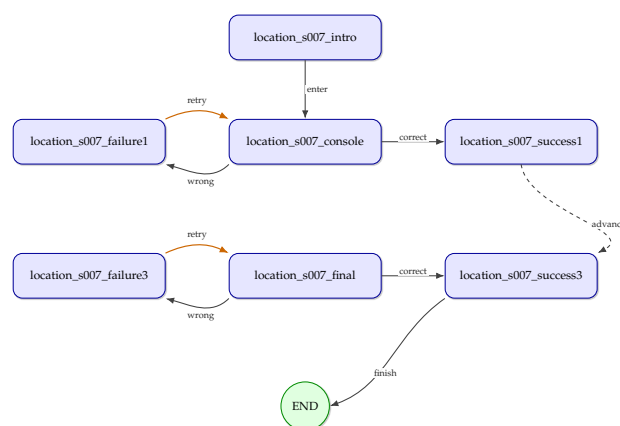


Figure A1. Reduced beat graph for the Ink example in Listing A1. Correct answers advance the main path, wrong answers enter local retry loops, and the dashed edge condenses omitted intermediate progression.

Appendix B. Ink GBNF Grammar

Listing A2. Ink GBNF grammar for strategy S4.

```

root ::= start

// NOTE:
// This file is used for human-readable documentation in the paper appendix.
// Character classes below use ASCII hex escapes:
// \x09 = TAB, \x20 = space, \x3D = '=', \x7B = '{', \x7C = '|', \x7E = '~'

start ::= think fenced-ink

fenced-ink ::= fence-open ink fence-close

fence-open ::= "`" "ink" nl
fence-close ::= "`"

think ::= "<think>" think-content "</think>" nl+
think-content ::= think-char*
think-char ::= [^<]

ink ::= startdivert knots endmarker

startdivert ::= "->" wsp+ knotname nl+

knots ::= knot (nl+ knot)*

knot ::= knotheader nl knotbody

knotheader ::= "===" wsp* knotname wsp* "===" tagsopt?

tagsopt ::= wsp+ tags

tags ::= tag (wsp+ tag)*
tag ::= "#" tagname
tagname ::= tagchar tagname | tagchar
tagchar ::= [A-Za-z0-9_]

knotbody ::= line (line)*

line ::= choiceline | gatherline | divertline | tagline | textline | blankline

choiceline ::= wsp* choicebullet wsp* "[" choicetext "]" choicedivertopt? nl
choicebullet ::= "*" | "+"
choicetext ::= choicechar choicetext | choicechar
// Printable ASCII for choice text, with selected exclusions via split ranges.
choicechar ::= [\x20-\x5C\x5E-\x7A\x7C-\x7E]
choicedivertopt ::= wsp* "->" wsp* knottarget

gatherline ::= wsp* "-" wsp+ textinline tagsopt? nl
textinline ::= textchar texttail?

divertline ::= wsp* "->" wsp* knottarget nl

tagline ::= wsp* tags nl

textline ::= wsp* textstart texttail? tagsopt? nl
// textstart allows TAB and printable ASCII, but excludes '=' (\x3D).
// This avoids ambiguity with knot headers like "=== ... ===".
textstart ::= [\x09\x20-\x3C\x3E-\x7E]
texttail ::= textchar texttail | textchar
// textchar allows TAB and printable ASCII, but excludes '{' (\x7B).
// This keeps the accepted text subset conservative for this grammar profile.
textchar ::= [\x09\x20-\x7A\x7C-\x7E]

blankline ::= wsp* nl

knottarget ::= knotname | "END"
knotname ::= nameseg ("_" nameseg)+
nameseg ::= namefirst namechar*
namefirst ::= [A-Za-z]
namechar ::= [A-Za-z0-9_]

endmarker ::= wsp* "#end_of_file" nl

wsp ::= " " | "\t"
nl ::= "\n"

```


References

1. Maxim, R.I.; Arnedo-Moreno, J. Identifying Key Principles and Commonalities in Digital Serious Game Design Frameworks: Scoping Review. *JMIR Serious Games* **2025**, *13*, e54075. [\[CrossRef\]](#)
2. Horn, F.; Göbel, S. AI as a Co-creator: A Survey on AI Support for Educational Game Authoring Tools. In *Proceedings of the Joint International Conference on Serious Games (JCSG 2024)*; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2025; Volume 15259, pp. 3–18. [\[CrossRef\]](#)
3. Humble, N. Play My Thesis: Opportunities and Limitations of Building Serious Games with AI. In *Proceedings of the International Conference on AI Research*, Lisbon, Portugal, 5–6 December 2024; Volume 4, pp. 165–174. [\[CrossRef\]](#)
4. Brandl, L.C.; Schrader, A. Realizing Ambient Serious Games in Higher Education—Concept and Heuristic Evaluation. *Trends High. Educ.* **2025**, *4*, 52. [\[CrossRef\]](#)
5. Gallotta, R.; Todd, G.; Zammit, M.; Earle, S.; Liapis, A.; Togelius, J.; Yannakakis, G.N. Large Language Models and Games: A Survey and Roadmap. *IEEE Trans. Games* **2024**, 1–18. [\[CrossRef\]](#)
6. Browne, C.; Maire, F. Evolutionary game design. *IEEE Trans. Comput. Intell. AI Games* **2010**, *2*, 1–16. [\[CrossRef\]](#)
7. Summerville, A.; Martens, C.; Samuel, B.; Osborn, J.; Wardrip-Fruin, N.; Mateas, M. Gemini: Bidirectional generation and analysis of games via ASP. In *Proceedings of the AAAI Conference on Artificial Intelligence (AAAI)*, New Orleans, LA, USA, 2–7 February 2018.
8. Khalifa, A.; Green, M.C.; Perez-Liebana, D.; Togelius, J. General video game rule generation. In *Proceedings of the IEEE Conference on Computational Intelligence and Games (CIG)*, New York, NY, USA, 22–25 August 2017.
9. Maurer, T.; Guzdial, M. Adversarial random forest classifier for automated game design. In *Proceedings of the Foundations of Digital Games (FDG)*, Montreal, QC, Canada, 3–6 August 2021.
10. Guzdial, M.; Riedl, M.O. Conceptual game expansion. *IEEE Trans. Games* **2022**, *14*, 93–106. [\[CrossRef\]](#)
11. Todd, G.; Earle, S.; Nasir, M.U.; Green, M.C.; Togelius, J. Level generation through large language models. In *Proceedings of the Foundations of Digital Games (FDG)*, Lisbon, Portugal, 12–14 April 2023.
12. Sudhakaran, S.; González-Duque, M.; Freiburger, M.; Glanois, C.; Najarro, E.; Risi, S. MarioGPT: Open-ended text2level generation through large language models. In *Proceedings of the NeurIPS*, New Orleans, LA, USA, 10–16 December 2023.
13. Brown, T.; Mann, B.; Ryder, N.; Subbiah, M.; Kaplan, J.D.; Dhariwal, P.; Neelakantan, A.; Shyam, P.; Sastry, G.; Askell, A.; et al. Language models are few-shot learners. In *Proceedings of the NeurIPS*, Vancouver, BC, Canada, 6–12 December 2020.
14. Liu, P.; Yuan, W.; Fu, J.; Jiang, Z.; Hayashi, H.; Neubig, G. Pre-Train, Prompt, and Predict: A Systematic Survey of Prompting Methods in Natural Language Processing. *ACM Comput. Surv.* **2023**, *55*, 1–35. [\[CrossRef\]](#)
15. Yang, D.; Kleinman, E.; Hartevelde, C. GPT for Games: An Updated Scoping Review (2020–2024). *arXiv* **2024**. [\[CrossRef\]](#)
16. Zhou, E.; Basavatia, S.; Siam, M.; Chen, Z.; Riedl, M.O. STORY2GAME: Generating (Almost) Everything in an Interactive Fiction Game. *arXiv* **2025**, arXiv:2505.03547. [\[CrossRef\]](#)
17. Nasir, M.U.; James, S.; Togelius, J. Word2world: Generating stories and worlds through large language models. *arXiv* **2024**, arXiv:2405.06686.
18. Schaul, T. A Video Game Description Language for Model-Based or Interactive Learning. In *Proceedings of the 2013 IEEE Conference on Computational Intelligence in Games (CIG)*; IEEE: Piscataway, NJ, USA, 2013; pp. 1–8. [\[CrossRef\]](#)
19. Hu, C.; Zhao, Y.; Liu, J. Game generation via large language models. In *Proceedings of the 2024 IEEE Conference on Games (CoG)*; IEEE: Piscataway, NJ, USA, 2024; pp. 1–4.
20. Tanaka, T.; Simo-Serra, E. Grammar-based Game Description Generation using Large Language Models. *IEEE Trans. Games* **2024**, *18*, 30–43. [\[CrossRef\]](#)
21. Piette, É.; Soemers, D.; Stephenson, M.; Sironi, C.; Winands, M.; Browne, C. Ludii—The Ludemic General Game System. In *Proceedings of the ECAI 2020*; De Giacomo, G., Catala, A., Dilkina, B., Milano, M., Barro, S., Bugarin, A., Lang, J., Eds.; Frontiers in Artificial Intelligence and Applications; IOS Press: Amsterdam, The Netherlands, 2020; Volume 325, pp. 411–418. [\[CrossRef\]](#)
22. Anjum, A.; Li, Y.; Law, N.; Charity, M.; Togelius, J. The Ink Splotch Effect: A Case Study on ChatGPT as a Co-Creative Game Designer. In *Proceedings of the 19th International Conference on the Foundations of Digital Games; FDG '24*; Association for Computing Machinery: New York, NY, USA, 2024. [\[CrossRef\]](#)
23. Lanzi, P.L.; Loiacono, D. ChatGPT and Other Large Language Models as Evolutionary Engines for Online Interactive Collaborative Game Design. In *Proceedings of the Genetic and Evolutionary Computation Conference*; ACM: New York, NY, USA, 2023; pp. 1383–1390. [\[CrossRef\]](#)
24. Ammanabrolu, P.; Cheung, W.; Tu, D.; Broniec, W.; Riedl, M. Bringing Stories Alive: Generating Interactive Fiction Worlds. *Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entertain.* **2020**, *16*, 3–9. [\[CrossRef\]](#)
25. Sun, Y.; Li, Z.; Fang, K.; Lee, C.H.; Asadipour, A. Language as Reality: A Co-Creative Storytelling Game Experience in 1001 Nights Using Generative AI. *Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entertain.* **2023**, *19*, 425–434. [\[CrossRef\]](#)
26. Farrell, R.; Ware, S.G. Large Language Models as Narrative Planning Search Guides. *IEEE Trans. Games* **2025**, *17*, 419–428. [\[CrossRef\]](#)

27. Van Stegeren, J.; Myśliwiec, J. Fine-Tuning GPT-2 on Annotated RPG Quests for NPC Dialogue Generation. In *Proceedings of the 16th International Conference on the Foundations of Digital Games (FDG) 2021*; ACM: New York, NY, USA, 2021; pp. 1–8. [\[CrossRef\]](#)
28. Ashby, T.; Webb, B.K.; Knapp, G.; Searle, J.; Fulda, N. Personalized Quest and Dialogue Generation in Role-Playing Games: A Knowledge Graph- and Language Model-based Approach. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems*; ACM: New York, NY, USA, 2023; pp. 1–20. [\[CrossRef\]](#)
29. Al-Nassar, S.; Schaap, A.; Zwart, M.V.D.; Preuss, M.; Gómez-Maureira, M.A. QuestVille: Procedural Quest Generation Using NLP Models. In *Proceedings of the 18th International Conference on the Foundations of Digital Games*; ACM: New York, NY, USA, 2023; pp. 1–4. [\[CrossRef\]](#)
30. Buongiorno, S.; Klinkert, L.; Zhuang, Z.; Chawla, T.; Clark, C. PANGeA: Procedural artificial narrative using generative AI for turn-based, role-playing video games. *Proc. AAAI Conf. Artif. Intell. Interact. Digit. Entertain.* **2024**, *20*, 156–166. [\[CrossRef\]](#)
31. Torii, M.G.; Murakami, T.; Ochiai, Y. Lottery and Sprint: Generate a Board Game with Design Sprint Method on AutoGPT. In *Proceedings of the CHI PLAY Companion*; ACM: New York, NY, USA, 2023; pp. 259–265. [\[CrossRef\]](#)
32. Tanaka, T.; Simo-Serra, E. Grammar and Gameplay-aligned RL for Game Description Generation with LLMs. In *Proceedings of the IEEE 2025 Conference on Games (CoG 2025)*, Lisbon, Portugal, 26–29 August 2025; pp. 1–8. [\[CrossRef\]](#)
33. Connolly, T.M.; Boyle, E.A.; MacArthur, E.; Hailey, T.; Boyle, J.M. A systematic literature review of empirical evidence on computer games and serious games. *Comput. Educ.* **2012**, *59*, 661–686. [\[CrossRef\]](#)
34. Laamarti, F.; Eid, M.; El Saddik, A. An Overview of Serious Games. *Int. J. Comput. Games Technol.* **2014**, *2014*, 358152. [\[CrossRef\]](#)
35. Riedl, M.O.; Bulitko, V. Interactive Narrative: An Intelligent Systems Approach. *AI Mag.* **2013**, *34*, 67–77. [\[CrossRef\]](#)
36. Robin, B.R. Digital Storytelling: A Powerful Technology Tool for the 21st Century Classroom. *Theory Pract.* **2008**, *47*, 220–228. [\[CrossRef\]](#)
37. AlHussain, A.I.; Azmi, A.M. Automatic Story Generation: A Survey of Approaches. *ACM Comput. Surv.* **2021**, *54*, 1–38. [\[CrossRef\]](#)
38. Hargood, C.; Weal, M.J.; Millard, D.E. The StoryPlaces Platform: Building a Web-Based Locative Hypertext System. In *Proceedings of the 29th on Hypertext and Social Media*; ACM: New York, NY, USA, 2018; pp. 128–135. [\[CrossRef\]](#)
39. Vrettakis, E.; Kourtis, V.; Katifori, A.; Karvounis, M.; Lougiakis, C.; Ioannidis, Y. Narralive—Creating and experiencing mobile digital storytelling in cultural heritage. *Digit. Appl. Archaeol. Cult. Herit.* **2019**, *15*, e00114. [\[CrossRef\]](#)
40. Trichopoulos, G.; Alexandridis, G.; Caridakis, G. A Survey on Computational and Emergent Digital Storytelling. *Heritage* **2023**, *6*, 1227–1263. [\[CrossRef\]](#)
41. Svensson, T.; Holloway-Attaway, L.; Beroldy, E. Leaving the Small Screen: Telling News Stories in a VR Simulation of an AR News Service. In *Proceedings of the Interactive Storytelling*; Rouse, R., Koenitz, H., Haahr, M., Eds.; Lecture Notes in Computer Science; Springer: Cham, Switzerland, 2018; Volume 11318, pp. 352–355. [\[CrossRef\]](#)
42. van Rozen, R. Languages of Games and Play: A Systematic Mapping Study. *ACM Comput. Surv.* **2021**, *53*, 1–37. [\[CrossRef\]](#)
43. Genesereth, M.; Love, N.; Pell, B. General Game Playing: Overview of the AAAI Competition. *AI Mag.* **2005**, *26*, 62–72. [\[CrossRef\]](#)
44. Kowalski, J.; Mika, M.; Sutowicz, J.; Szykuła, M. Regular Boardgames. *Proc. AAAI Conf. Artif. Intell.* **2019**, *33*, 1699–1706. [\[CrossRef\]](#)
45. Studios, I. Ink—Inkle’s Narrative Scripting Language. Available online: <https://www.inklestudios.com/ink/> (accessed on 20 May 2025).
46. Baker, S. Choose your own educational resource: Developing an interactive OER using the ink scripting language. *Code4Lib J.* **2021**, *51*. Available online: <https://journal.code4lib.org/articles/15721> (accessed on 16 March 2026).
47. Rogosch, F.; Schrader, A. *SINE Paper Companion Software and Data Release*, Version 1.0.0; Zenodo: Geneva, Switzerland, 2026. [\[CrossRef\]](#)
48. Ggml-Org/Llama.Cpp. Available online: <https://github.com/ggml-org/llama.cpp> (accessed on 16 December 2025).
49. Unsloth Model Catalog | Unsloth Documentation. Available online: <https://unsloth.ai/docs/get-started/unsloth-model-catalog> (accessed on 13 January 2026).
50. Ggml/Docs/Gguf.Md at Master · Ggml-Org/Ggml. Available online: <https://github.com/ggml-org/ggml/blob/master/docs/gguf.md> (accessed on 16 December 2025).
51. Hugging Face. The Model Hub. Available online: <https://huggingface.co/docs/hub/en/models-the-hub> (accessed on 16 February 2026).
52. Unsloth AI—Open Source Fine-Tuning & RL for LLMs. Available online: <https://unsloth.ai/> (accessed on 16 December 2025).
53. OpenAI. gpt-oss-120b & gpt-oss-20b Model Card. *arXiv* **2025**. [\[CrossRef\]](#)
54. IBM Research. Granite 4.0 Language Models. Available online: <https://github.com/ibm-granite/granite-4.0-language-models> (accessed on 13 February 2026).
55. Abdin, M.; Aneja, J.; Behl, H.; Bubeck, S.; Eldan, R.; Gunasekar, S.; Harrison, M.; Hewett, R.J.; Javaheripi, M.; Kauffmann, P.; et al. Phi-4 Technical Report. *arXiv* **2024**. [\[CrossRef\]](#)
56. DeepSeek-AI. DeepSeek-V3 Technical Report. *arXiv* **2024**. [\[CrossRef\]](#)

57. Team, G.; Kamath, A.; Ferret, J.; Pathak, S.; Vieillard, N.; Merhej, R.; Perrin, S.; Matejovicova, T.; Ramé, A.; Rivière, M.; et al. Gemma 3 Technical Report. *arXiv* **2025**. [[CrossRef](#)]
58. Meta. Llama 3. Available online: <https://www.llama.com/models/llama-3/> (accessed on 20 May 2025).
59. QwenTeam. Qwen3. Available online: <https://qwenlm.github.io/blog/qwen3/> (accessed on 13 February 2026).
60. Mistral-AI; Rastogi, A.; Jiang, A.Q.; Lo, A.; Berrada, G.; Lample, G.; Rute, J.; Barmantlo, J.; Yadav, K.; Khandelwal, K.; et al. Magistral. *arXiv* **2025**. [[CrossRef](#)]
61. Zhu, X.; Li, J.; Liu, Y.; Ma, C.; Wang, W. A Survey on Model Compression for Large Language Models. *Trans. Assoc. Comput. Linguist.* **2024**, *12*, 1556–1577. [[CrossRef](#)]
62. Leandro, J.; Rao, S.; Xu, M.; Xu, W.; Jojic, N.; Brockett, C.J.; Dolan, W.B. GENEVA: GENERating and Visualizing branching narratives using LLMs. In Proceedings of the 2024 IEEE Conference on Games (CoG), Milan, Italy, 5–8 August 2024; pp. 1–5.
63. Sun, Y.; Wang, P.J.; Chung, J.J.Y.; Roemmele, M.; Kim, T.; Kreminski, M. Drama Llama: An LLM-Powered Storylets Framework for Authorable Responsiveness in Interactive Narrative. *arXiv* **2025**. [[CrossRef](#)]
64. Netz, L.; Reimer, J.; Rumpe, B. Using Grammar Masking to Ensure Syntactic Validity in LLM-based Modeling Tasks. In *Proceedings of the ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems; MODELS Companion '24*; ACM: New York, NY, USA, 2024; pp. 115–122. [[CrossRef](#)]
65. Kamoi, R.; Zhang, Y.; Zhang, N.; Han, J.; Zhang, R. When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs. *Trans. Assoc. Comput. Linguist.* **2024**, *12*, 1417–1440. [[CrossRef](#)]
66. Szilas, N.; Ilea, I. Objective Metrics for Interactive Narrative. In *Interactive Storytelling*; Mitchell, A., Fernández-Vara, C., Thue, D., Eds.; Springer International Publishing: Berlin/Heidelberg, Germany, 2014; Volume 8832, pp. 91–102. [[CrossRef](#)]
67. Chang, Y.; Wang, X.; Wang, J.; Wu, Y.; Yang, L.; Zhu, K.; Chen, H.; Yi, X.; Wang, C.; Wang, Y.; et al. A Survey on Evaluation of Large Language Models. *ACM Trans. Intell. Syst. Technol.* **2024**, *15*, 1–45. [[CrossRef](#)]
68. Wang, Q.; Wang, M.; Feldhus, N.; Ostermann, S.; Cao, Y.; Schütze, H.; Möller, S.; Schmitt, V. Through a Compressed Lens: Investigating The Impact of Quantization on Factual Knowledge Recall. *arXiv* **2026**. [[CrossRef](#)]
69. Li, L.; Sleem, L.; Gentile, N.; Nichil, G.; State, R. Exploring the Impact of Temperature on Large Language Models: Hot or Cold? *Procedia Comput. Sci.* **2025**, *264*, 242–251. [[CrossRef](#)]
70. Arnab, S.; Lim, T.; Carvalho, M.B.; Bellotti, F.; De Freitas, S.; Louchart, S.; Suttie, N.; Berta, R.; De Gloria, A. Mapping learning and game mechanics for serious games analysis. *Br. J. Educ. Technol.* **2015**, *46*, 391–411.

Disclaimer/Publisher’s Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.